# Lab 08 — Vragen

### Vraag 1 [Begrip]

Een team bouwt drie images: `api:1.4.2-dev`, `api:1.4.2-staging` en `api:1.4.2-prod`, elk met zijn eigen `application.yaml` erin gekopieerd. De uitrol in productie faalt op een parameter die niemand getest had. Leg de methodefout uit, wat er had moeten gebeuren, en wat je precies verliest door één image per omgeving te bouwen.

### Vraag 2 [Voorspelling]

De image `api:env` bevat `ENV APP_PROFILE=image`. Het bestand `api.env` bevat `APP_PROFILE=staging`. Wat is de waarde van `APP_PROFILE` in elk van de volgende gevallen, en waarom speelt de volgorde van de opties geen rol?

```
podman run --env-file api.env api:env
podman run --env-file api.env -e APP_PROFILE=prod api:env
podman run -e APP_PROFILE=prod --env-file api.env api:env
APP_PROFILE=shell podman run -e APP_PROFILE api:env
```

### Vraag 3 [Diagnose]

```
$ podman run -d --name api -p 18080:8080 -e SERVER_PORT=9090 api:1.0
$ curl -s localhost:18080/actuator/health
$ podman logs api
API demarree sur le port 9090 (profil default)
```

De API draait, de logs zijn normaal, `curl` geeft niets terug. Wat klopt er niet, en wat zegt dit voorbeeld over de relatie tussen `EXPOSE`, `-p` en de configuratie van de applicatie?

### Vraag 4 [Analyse]

Een collega start `podman run -d --name api api:1.0 --db-password=Secr3t` "omdat de applicatie het wachtwoord als argument aanvaardt". Som **vier** plaatsen op waar dat wachtwoord leesbaar is, telkens met het commando dat het bewijst, en rangschik ze van ernstig naar minder ernstig.

### Vraag 5 [Analyse]

Dezelfde collega schakelt over op `-e DB_PASSWORD=Secr3t`. Is dat beter? Som de plaatsen op waar de waarde zichtbaar blijft, en leg dan uit wat de conventie `POSTGRES_PASSWORD_FILE` concreet verandert in `podman inspect`.

### Vraag 6 [Begrip]

Leg uit hoe Spring Boot `SPRING_DATASOURCE_URL` koppelt aan `spring.datasource.url`, wat `SPRING_PROFILES_ACTIVE=prod` doet, en in welke volgorde Spring Boot beslist bij een waarde die tegelijk in `application.yaml` (in de JAR), in `/config/application.yaml` (gemount) en in een omgevingsvariabele staat.

### Vraag 7 [Diagnose]

Een Dockerfile bevat:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
ENV SPRING_PROFILES_ACTIVE=prod
ENV SPRING_DATASOURCE_PASSWORD=Pr0d!
COPY .env /app/.env
COPY target/api.jar /app/api.jar
ENTRYPOINT ["java","-jar","/app/api.jar"]
```

Wijs regel per regel aan wat betwistbaar is, en stel de gecorrigeerde Dockerfile voor, met uitleg over waar elke verplaatste informatie dan wél thuishoort.

### Vraag 8 [Voorspelling]

```
printf 'S3cr3t' | podman secret create db_password -
podman run -d --name db --secret db_password -e POSTGRES_PASSWORD_FILE=/run/secrets/db_password
postgres:16-alpine
```

Wat tonen `podman inspect db --format '{{.Config.Env}}'`, `podman exec db ls -l /run/secrets/` en `podman secret inspect db_password`? En wat moet je aan `podman secret inspect` toevoegen om de waarde terug te lezen — wat bewijst dat?

### Vraag 9 [Analyse]

`podman secret` slaat de waarden base64-gecodeerd op in je gebruikersmap, zonder versleuteling. Een collega besluit daaruit dat "het nergens toe dient, dan kun je even goed `-e` gebruiken". Wat brengt `podman secret` toch bij ten opzichte van `-e`? En wat brengt een secretsbeheerder (Vault…) meer dan `podman secret`?

### Vraag 10 [Diagnose]

```
$ podman build --secret id=token,src=token.txt -t app .      # OK
$ rm token.txt
$ podman build -t app .                                      # ook OK?!
```

De Dockerfile bevat `RUN --mount=type=secret,id=token,required=true npm ci`. Waarom slaagt de tweede build zonder secret, en wat is er nodig om hem te zien falen? Welk praktisch gevolg heeft dit voor een token dat verloopt?

### Vraag 11 [Begrip]

Vergelijk `ARG NPM_TOKEN` + `RUN npm ci` met `RUN --mount=type=secret,id=npm_token npm ci`: waar belandt het token in elk geval, en met welk commando ga je dat na op de geproduceerde image? Zou de multi-stage van lab 05 volstaan om de `ARG` te beschermen?

### Vraag 12 [Analyse]

Een bestand `.env` met de ontwikkelwachtwoorden wordt in Git gecommit "omdat het toch maar dev is". Geef drie verschillende redenen waarom dat ook in ontwikkeling een probleem is, en de vervangende praktijk (bestanden, `.gitignore`, `.dockerignore`, gecommit voorbeeld).

### Vraag 13 [Analyse]

In productie is een databasesecret gelekt in een supportticket (iemand plakte de uitvoer van `podman inspect`). Beschrijf stap voor stap de verwachte reactie, en leg uit waarom "de opmerking

uit het ticket verwijderen" daar geen deel van uitmaakt — en wat in de architectuur die reactie snel of traag maakt.